

Stack and Queue

by Harsh Jain

1) Implementation of Stack

```
#include <bits/stdc++.h>
using namespace std;

struct Mystack{
    int top;
    int cap;
    int *arr;
    Mystack(int c){
        cap = c;
        top = -1;
        arr = new int[cap];
    }

    void push(int x){
        if(top == cap-1){
            cout << "Stack overflow" << endl;
            return;
        }
        top++;
        arr[top] = x;
    }

    int pop(){
        if(top == -1){
            cout << "Stack underflow" << endl;
            return -1;
        }
        int ele = arr[top];
```

```

        top--;
        return ele;
    }

    int peek(){
        if(top==-1){
            cout << "Stack is empty" << endl;
            return -1;
        }
        return arr[top];
    }

    int size(){
        return top+1;
    }

    bool isEmpty(){
        return top == -1;
    }
};

int main(){
    Mystack s(5);
    s.push(10);
    s.push(20);
    cout << s.size() << endl;
    cout << s.peek() << endl;
    s.pop();
    cout << s.peek() << endl;
}

```

2)Implementation of Queue

In this program our all the functions are $O(1)$ Time Complexity but dequeue is $O(n)$ Time

```
#include <bits/stdc++.h>
using namespace std;

struct MyQueue{
    int cap;
    int top;
    int *arr;
    MyQueue(int c){
        cap = c;
        top = -1;
        arr = new int[cap];
    }

    void enqueue(int x){
        top++;
        arr[top] = x;
    }

    int dequeue(){
        int temp = arr[0];
        for(int i=1; i<=top; i++){
            arr[i-1] = arr[i];
        }
        top--;
        return temp;
    }

    int getFront(){
        return arr[0];
    }

    int getRear(){
        return arr[top];
    }
}
```

```
int size(){
    return top+1;
}

bool isEmpty(){
    return top==-1;
}
};

int main(){
    MyQueue q(5);
    q.enqueue(10);
    q.enqueue(20);
    cout << q.dequeue() << endl;
    cout << q.dequeue() << endl;
}
```

3)Optimized Implementation of Queue

In this program we used the concept of Circular Array

```
#include <bits/stdc++.h>
using namespace std;

// Optimized Queue

struct MyQueue{
    int *arr;
    int front;
    int size;
    int cap;
    MyQueue(int c){
        cap = c;
        front = 0;
        size = 0;
        arr = new int[cap];
    }

    bool isFull(){
        return size == cap;
    }

    bool isEmpty(){
        return size == 0;
    }

    int getFront(){
        if(size == 0){
            cout << "Queue is empty";
            return -1;
        }
        return arr[front];
    }
}
```

```
void enqueue(int x){
    if(size == cap){
        cout << "queue is full" << endl;
        return ;
    }
    int idx = (front+size)%cap;
    arr[idx] = x;
    size++;
}

int dequeue(){
    if(size == 0){
        cout << "Queue is empty" << endl;
        return -1;
    }
    int ele = arr[front];
    front = (front+1)%cap;
    size--;
    return ele;
}

};
```

```
int main(){
    MyQueue q(5);
    q.enqueue(10);
    q.enqueue(20);
    cout << q.dequeue() << endl;
    cout << q.dequeue() << endl;
}
```

4) Stack in C++ STL

<https://www.geeksforgeeks.org/stack-in-cpp-stl/>

5) Queue in C++ STL

<https://www.geeksforgeeks.org/queue-cpp-stl/>

6) Design Stack and Queue using LinkedList

Questions

Q1) Valid Parentheses

Link: <https://leetcode.com/problems/valid-parentheses/>

Code:

```
bool isValid(string s) {
    int n = s.size();
    stack<char>st;
    for(int i=0; i<n; i++){
        if(s[i] == '(' || s[i] == '{' || s[i] == '['){
            st.push(s[i]);
        }
        else{
            if(st.empty() == true) return false;
            char topp = st.top();
            st.pop();
            if( (s[i] == ')' and topp == '(') ||
                (s[i] == '}' and topp == '{') ||
                (s[i] == ']' and topp == '[')
            ){
                continue;
            }
            else{
                return false;
            }
        }
    }
    if(st.empty() == false) return false;
    return true;
}
```


Q2) Set of 4 Question

- a) Previous greater element
- b) Previous smaller element
- c) Next greater element
- d) Next Smaller element

Code:

```
#include <bits/stdc++.h>
using namespace std;

void previousSmallerEle(vector<int>arr) {
    int n = arr.size();
    for(int i=0; i<n; i++){
        int j=i-1;
        for(; j>=0; j--){
            if(arr[j] < arr[i]){
                break;
            }
        }
        if(j==-1) cout << -1 << " ";
        else cout << arr[j] << " ";
    }
}

//optimized code

void previousSmallerEle(vector<int>arr) {
    int n = arr.size();
    stack<int>s;
    for(int i=0; i<n; i++){
        while(s.empty() == false and s.top() >= arr[i]){
            s.pop();
        }
        if(s.empty() == true) cout << -1 << " ";
        else cout << s.top() << " ";
        s.push(arr[i]);
    }
}
```

```

// Previous Greater element
void previousSmallerEle(vector<int>arr){
    int n = arr.size();
    stack<int>s;
    for(int i=0; i<n; i++){
        while(s.empty() == false and s.top() <= arr[i]){
            s.pop();
        }
        if(s.empty() == true) cout << -1 << " ";
        else cout << s.top() << " ";
        s.push(arr[i]);
    }
}

// Next Greater element
// in this code element will be printed in reverser order so put them
// in an array and reverse them
void previousSmallerEle(vector<int>arr){
    int n = arr.size();
    stack<int>s;
    for(int i=n-1; i>=0; i--){
        while(s.empty() == false and s.top() <= arr[i]){
            s.pop();
        }
        if(s.empty() == true) cout << -1 << " ";
        else cout << s.top() << " ";
        s.push(arr[i]);
    }
}

// Next Greater element
// in this code element will be printed in reverser order so put them
// in an array and reverse them

```

```
void previousSmallerEle(vector<int>arr) {
    int n = arr.size();
    stack<int>s;
    for(int i=n-1; i>=0; i--){
        while(s.empty() == false and s.top() >= arr[i]){
            s.pop();
        }
        if(s.empty() == true) cout << -1 << " ";
        else cout << s.top() << " ";
        s.push(arr[i]);
    }
}

int main(){
    vector<int>arr = {1,2,3,4,5};
    previousSmallerEle(arr);
}
```

Q3) Largest area in Histogram

Link: <https://leetcode.com/problems/largest-rectangle-in-histogram/>

Code:

```
vector<int> previousSmaller(vector<int>&arr) {
    int n = arr.size();
    vector<int>ans;
    stack<int>s;
    for(int i=0; i<n; i++){
        while(s.empty() == false and arr[s.top()] >=
arr[i]){
            s.pop();
        }
        if(s.empty()){
            ans.push_back(-1);
        }
        else{
            ans.push_back(s.top());
        }
        s.push(i);
    }
    return ans;
}

vector<int> nextSmaller(vector<int>&arr) {
    int n = arr.size();
    vector<int>ans;
    stack<int>s;
    for(int i=n-1; i>=0; i--){
        while(s.empty() == false and arr[s.top()] >=
arr[i]){
            s.pop();
        }
        if(s.empty()){
            ans.push_back(n);
        }
        else{
            ans.push_back(s.top());
        }
        s.push(i);
    }
}
```

```
        }  
        reverse(ans.begin(),ans.end());  
        return ans;  
    }  
    int largestRectangleArea(vector<int>& heights) {  
        vector<int>ps = previousSmaller(heights);  
        vector<int>ns = nextSmaller(heights);  
        int ans = 0;  
        int n = heights.size();  
        for(int i=0; i<n; i++){  
            int len = ns[i]-ps[i]-1;  
            int area = heights[i]*len;  
            ans = max(ans,area);  
        }  
        return ans;  
    }  
}
```

Q4) Trapping Rainwater

Link: <https://leetcode.com/problems/trapping-rain-water/>

Video Link: <https://youtu.be/m18Hntz4go8>

Q5) Maximal Rectangles

Link: <https://leetcode.com/problems/maximal-rectangle/>

Q6) Online Stock Span / Stock Span Problem

Link: <https://leetcode.com/problems/online-stock-span/>

Q7) Implement Queue using 2 stack**Q8) Implement Stack using 2 Queue****Q9) Implement 2 Stack in One Array****Q10) Stack with getMin() in O(1)****Q11) Asteroid Collision**

Link: <https://leetcode.com/problems/asteroid-collision/>

Q12) Remove K Digits

Link: <https://leetcode.com/problems/remove-k-digits/>

Q13) Design LRU Cache**Q14) Learn about infix, prefix and postfix expressions not need to code****Q15) Reversing a Queue****Q16) Reversing a Stack****Q17) Implementation of dequeue : doubly ended Queue****Q18) Learn about STL of Dequeue****Q19) First Circular Tour**